JavaScriptのNumberは1つじゃない？

V8内部の型システムを覗く

西 悠太　/ 株式会社ダイニー

自己紹介

西 悠太 (Nishi Yuta)

V8 Contributor

Platform Engineer at Dinii Inc.

TSKaigi Staff

TypeScriptが好きです。
V8も好きです。

@riya-amemiya



今日のゴール

同じ number に見える値が、
V8 の中では 別の姿 で扱われることを知る。

まずは
JavaScriptとしてのNumber

仕様上はシンプル

JavaScriptの数値は、基本的には Number です。

```
let a: number = 42
let b: number = 4.2
let c: number = NaN
let d: number = Infinity
```

TypeScript で見ても、どれも **number** です。
整数、小数、NaN、Infinity を別の型としては扱いません。

早速、質問です

```
console.log(typeof 42)
console.log(typeof 4.2)
console.log(typeof NaN)
```

これは何が表示されるでしょう。

```
console.log(typeof 42)   // "number"
console.log(typeof 4.2)  // "number"
console.log(typeof NaN)  // "number"
```

42 も 4.2 も NaN も、すべて `"number"` です。

でも、V8から見ると？

42

Smi

小さな整数です。
値そのものを即値として埋め込みます。

4.2

HeapNumber

double 値です。
ヒープ上のオブジェクトとして持ちます。

JavaScript 上はどちらも `number` です。内部表現は違います。

V8の値は
Tagged Value で表現される

Tagged Value

V8 は JavaScript の値を、内部的には Tagged Value として扱います。

Smi	`… integer payload … 0`
HeapObject	`… heap address / offset … 1`

末尾のビットをタグとして使い、
小さな整数なのか、ヒープ上のオブジェクトへの参照なのかを区別します。

Smi: Small Integer

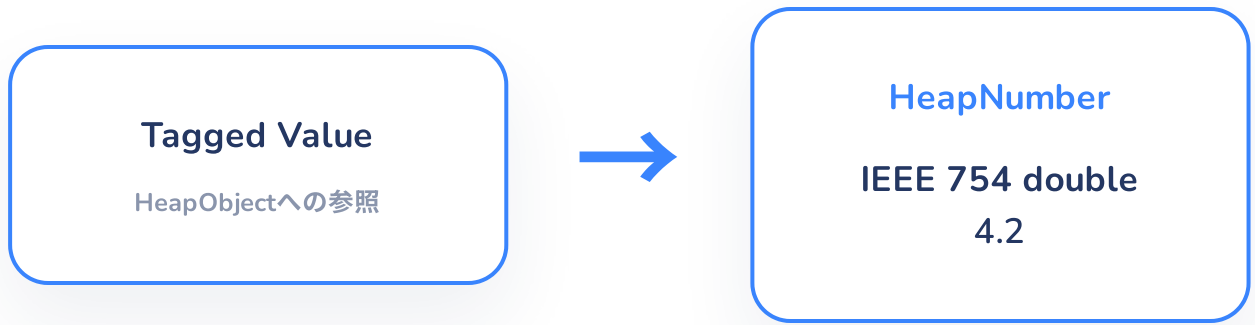
42

ヒープに置かず、値そのものを持つ

Small Integer = Smi

整数のたびに HeapNumber を作ると高コストです。
よく出る小さな整数は、Tagged Value の中に直接埋め込みます。

HeapNumber: ヒープ上のdouble



小数、NaN、Infinity、Smi に収まらない整数は、
HeapNumber として表現されます。

なぜ分けるのか

Smi にできると嬉しい

- → ヒープ割り当てがいらない
- → GC 対象が増えない
- → 整数演算として扱いやすい

HeapNumber が必要な理由

- → 小数を表現できる
- → NaN、Infinity、-0 を表現できる
- → Number 仕様の広い範囲をカバーできる

速く、小さく扱える値は Smi に載せ、
それでは足りない値は HeapNumber にします。

配列に入ると
さらに姿が変わる

V8は配列の中身も見ている

```
const a = [1, 2, 3]
const b = [1, 2, 3.14]
const c = [1, 2, "3"]
```

JavaScript から見ると、どれもただの `Array` です。
V8 は配列の要素に応じて Elements Kind を変えます。

Elements Kind

PACKED_SMI_ELEMENTS

```
[1, 2, 3]
```

Smi だけです。かなり特化できます。

PACKED_DOUBLE_ELEMENTS

```
[1, 2, 3.14]
```

double を生の値として並べられます。

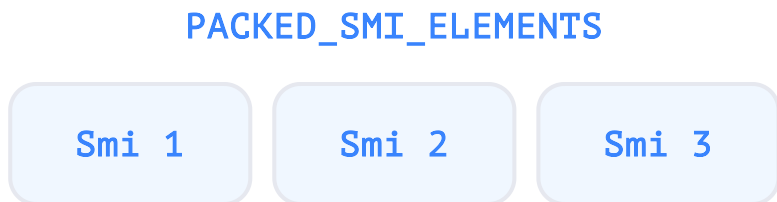
PACKED_ELEMENTS

```
[1, 2, "3"]
```

任意の JS 値です。数値は HeapNumber 参照になりえます。

整数だけの配列

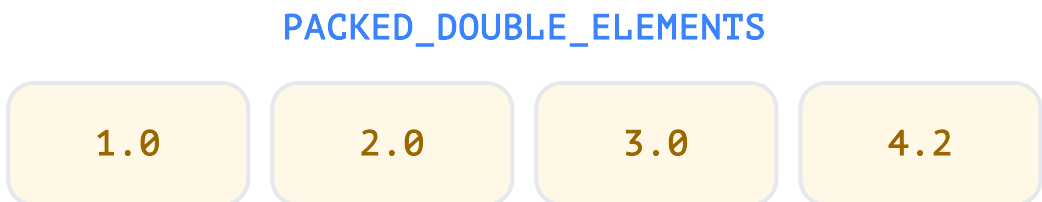
```
const xs = [1, 2, 3]
```



小さな整数だけなら、要素は Smi として扱えます。
要素ごとに HeapNumber を作る必要がありません。

小数が入った配列

```
const xs = [1, 2, 3]
xs.push(4.2)
```



小数が混ざると、Smi 配列から double 配列へ移ります。
この場合、配列の backing store には double 値を直接並べられます。

文字列が入った配列

```
const xs = [1, 2, 3]
xs.push(4.2)
xs.push("5")
```

PACKED_ELEMENTS

Smi 1	Smi 2	Smi 3	HeapNumber*	String*

任意の JS 値が混ざると、より一般的な要素表現へ移ります。
数値も、HeapNumber への参照として扱われる場面が出てきます。

遷移は基本、一方向



一度より一般的な Elements Kind へ遷移すると、
同じ配列インスタンスでは基本的に元の特化表現へ戻りません。

PACKED と HOLEY

PACKED

```
const xs = [1, 2, 3]
```

隙間がありません。
V8 が攻めた最適化をしやすいです。

HOLEY

```
const xs = [1, , 3]
xs[9] = 10
```

穴があります。
追加チェックやプロトタイプ探索が必要になります。

-0 や NaN も double 配列へ移す

```
const xs = [1, 2, 3] // PACKED_SMI_ELEMENTS

xs.push(-0)      // ここで PACKED_DOUBLE_ELEMENTS へ
xs.push(NaN)     // すでに double 配列
xs.push(Infinity) // すでに double 配列
```

見た目は整数に近くても、
Smi では表現できない Number は double 扱いになります。

実際に
V8の中を覗いてみる

DebugPrint

V8の `d8` では、デバッグ用の内部関数で
オブジェクトの内部表現を確認できます。

```
d8 --allow-natives-syntax
```

```
const xs = [1, 2, 3]
%DebugPrint(xs)
```

出力の `elements` に、
`PACKED_SMI_ELEMENTS` のような Elements Kind が表示されます。

trace-elements-transitions

```
const xs = [1, 2, 3]
xs.push(4.2)
xs.push("5")
```

```
d8 --trace-elements-transitions example.js
```

Elements Kind が変わったタイミングをログで追えます。
配列の表現が途中で変わる瞬間が見えます。

TypeScriptの型とは違うレイヤー

TypeScript

`number`

ECMAScript

`Number`

V8 Runtime

`Smi / HeapNumber / Elements Kind`

Machine Representation

`tagged value / double / pointer`

じゃあ普段のコードで
何を意識する？

まず、普通に書けばいい

V8 のために、無理なマイクロ最適化はしません。

V8 はかなり賢いです。
内部表現を知る目的は、普段のコードを全部 Smi に寄せることではありません。

ただし、大量データでは性能に効く

避けたい揺れ

- → 数値配列に突然文字列を混ぜる
- → 密な配列を不用意に疎にする
- → NaN や Infinity が大量に混ざる前提を放置する

安定しやすい形

- → 同じ意味の値は同じ表現に寄せる
- → 配列はできるだけ密に保つ
- → 数値の大量処理なら TypedArray も検討する

今日覚えて帰ってほしいこと

- ✓ JavaScript**上は**Numberでも、V8内部では表現が分かれる
- ✓ **小さな整数は** Smi、小数などは HeapNumber になりうる
- ✓ 配列では Elements Kind によって格納方法が変わる
- ✓ V8は「速く扱える形」を選ぶために、内部で細かく分類している

参考資料

- V8 Blog: Elements kinds in V8
- V8 Blog: Pointer Compression in V8
- V8 Blog: Fast properties in V8
- V8 Blog: The story of a V8 performance cliff in React

V8 の内部実装はバージョンやビルド設定、アーキテクチャによって変わります。
このスライドでは、代表的な概念に絞って説明しています。

ご清聴ありがとうございました

JavaScriptのNumberは、

V8の中でいくつもの姿に変わる。